NAMAL UNIVERSITY MIANWALI

Department of Computer Science

Database Design Document

CCP - Milestone 2

CattleCare Pro: Livestock Breeding and Farm Management Database System

Subject: CSC-271 – Database Systems

No.	Name	Roll No.
1	Muhammad Ali	NUM-BSCS-2024-46
2	Muhammad Ahmad	NUM-BSCS-2024-45
3	Aliya Ashraf	NUM-BSCS-2024-08

Submitted To:
Ms. Asiya Batool
Lecturer, Department of Computer Science

Submission Date: June 14, 2026

REVISION HISTORY

Date	Version	Description	Approved By
21/06/26	V 2.0	Added Chapter 3 (Logical Design) and Chapter 4 (Implementation): relational schema, functional dependencies, normalization, stored procedures, views, triggers, and GUI implementation details.	Ms. Asiya Batool
15/05/26	V 1.0	Initial submission. Includes project overview, entity identification, data dictionary, business rules, and EERD.	Ms. Asiya Batool

Contents

CHAPTER 1: Project Overview

1.1 INTRODUCTION

Livestock farming constitutes a significant pillar of the agricultural economy in Pakistan and across many developing nations. Effective management of cattle records, breeding history, and financial transaction data is essential for sustainable and profitable farm operations. Despite this importance, the majority of livestock farms continue to rely on manual record-keeping practices, including paper registers, handwritten ledgers, and basic spreadsheets. These traditional approaches introduce numerous inefficiencies: data loss due to physical damage or misplacement, inconsistency caused by human error, and extreme difficulty in retrieving or analysing historical records.

CattleCare Pro is a relational database-driven system designed specifically for livestock breeding and farm management. The system aims to replace fragmented manual methods with a structured, normalised, and query-efficient SQL database. It maintains comprehensive records of cattle identification, breeding lineage, birth and death events, purchase and sale transactions, periodic weight and colour tracking, and financial profit or loss calculations. By centralising all farm data into a single coherent relational schema, CattleCare Pro enables farm owners to make evidence-based decisions, trace complete animal histories, and manage their operations with greater precision and accountability.

The system is built around a carefully designed Enhanced Entity-Relationship (EERD) model that captures both the common attributes of all cattle and the specialised attributes of different cattle subtypes, namely cows and bulls. This EERD forms the foundation of the database schema implemented in MySQL, ensuring referential integrity, minimal redundancy, and scalability as farm data grows over time.

1.2 PROBLEM STATEMENT

Many livestock farm operators face serious challenges when relying on manual or informal systems for cattle management. The core problems identified are as follows:

- Difficulty maintaining complete and accurate cattle identification records across large herds.
- Lack of organised and searchable breeding history, making genetic lineage tracking nearly impossible.
- Inability to efficiently track birth events and link newborns to their parents in a structured manner.
- Poor management of cattle death records, resulting in outdated inventory and incorrect financial calculations.
- Disorganised purchase and sale transaction logs that complicate auditing and profit determination.
- Absence of systematic weight and colour tracking, which are important indicators for breeding selection and market valuation.
- No structured mechanism to compute and record profit or loss per animal after sale.

- Inefficient retrieval of historical records when required for inspections, loans, or insurance claims.

The cumulative effect of these problems results in poor financial performance visibility, breeding decisions based on incomplete information, and an overall inability to scale farm operations. A properly designed relational database system is therefore essential to address these limitations systematically.

1.3 PROJECT OBJECTIVES

The primary objective of CattleCare Pro is to design and implement a normalised relational SQL database that enables efficient storage, retrieval, and management of all livestock breeding and farm management data.

Specific objectives include:

- Design a fully normalised relational database schema for comprehensive livestock data management.
- Store detailed cattle identification records including tag number, breed, sex, date of birth, and current status.
- Implement a supertype–subtype structure to differentiate cows and bulls while sharing common cattle attributes.
- Maintain complete breeding records linking cows and bulls to each breeding event.
- Track birth records and associate each newborn with its breeding event and mother.
- Record death information for cattle that expire, preserving historical inventory accuracy.
- Manage purchase and sale transactions with complete pricing information.
- Store periodic weight and colour observations for each animal.
- Automatically compute and store profit or loss figures derived from purchase and sale records using database triggers.
- Provide a GUI front-end built in React and a REST API backend built in Node.js/Express for user interaction with the database.

GitHub Repository Link:
`https://github.com/aligoharcodepulse/cattlecare-pro-db.git`

CHAPTER 2: Conceptual Database Design

2.1 ENTITY

The following table identifies all entities present in the CattleCare Pro system. Each entity represents a distinct real-world object or concept that requires persistent storage within the database. The *Cattle* entity serves as a supertype, with *Cow* and *Bull* as its subtypes under an exclusive, total specialisation.

Table 2.1: Entity Identification

Sr. No.	Entity Name	Description
01	User	Farm administrator or operator who has login credentials and manages all farm data.
02	Farm	Physical farm or livestock unit registered in the system, owned and managed by a user.
03	Cattle	Supertype entity representing any animal on the farm, storing common attributes shared by all cattle.
04	Cow	Subtype of Cattle. Represents female cattle with attributes specific to their reproductive role.
05	Bull	Subtype of Cattle. Represents male cattle with attributes specific to their role as sires in breeding.
06	Breeding Record	Records a breeding event between a cow and a bull, capturing when and how the breeding occurred.
07	Birth Record	Records the birth of a calf, linking the newborn to its mother and breeding event.
08	Death Record	Stores mortality information for cattle that have died, including date and cause.
09	Purchase Record	Documents the purchase of a cattle animal, capturing vendor details, date, and purchase price.
10	Sale Record	Documents the sale of a cattle animal, capturing buyer details, date, and sale price.
11	Weight Record	Stores periodic weight measurements for a cattle animal to track growth over time.
12	Color Record	Records colour observations of a cattle animal used for identification and breeding selection.
13	Profit/Loss Record	Derived financial record that computes and stores the profit or loss from the sale of an animal.

2.2 DATA DICTIONARY

The data dictionary below provides a comprehensive definition of all attributes for each entity. It specifies attribute names, data types, constraints, and descriptive notes.

User

Table 2.2: User Entity Attributes

Sr.	Attribute	Data Type	Constraint	Description
01	UserID *(PK)*	INT	NOT NULL, AUTO_INC, PK	Unique system-generated identifier for each user account.
02	FullName	VARCHAR	NOT NULL	Full legal name of the user.
03	Email	VARCHAR	NOT NULL, UNIQUE	Email address used for login. Must be unique across all users.
04	Password	VARCHAR	NOT NULL	Hashed password string for secure authentication.
05	Role	ENUM	NOT NULL, DEFAULT ‘admin’	Role of the user: ‘admin’ or ‘viewer’.
06	Phone	VARCHAR	NULL	Optional contact phone number for the user.

Farm

Table 2.3: Farm Entity Attributes

Sr.	Attribute	Data Type	Constraint	Description
01	FarmID *(PK)*	INT	NOT NULL, AUTO_INC, PK	Unique identifier for each registered farm.
02	FarmName	VARCHAR	NOT NULL	Name of the farm as registered in the system.
03	Location	VARCHAR	NOT NULL	Physical address or geographic location of the farm.
04	Size	DECIMAL	NULL	Total area of the farm in acres or hectares.
05	OwnerID *(FK)*	INT	NOT NULL	Contact Info of the Farm Owner.

Cattle (Supertype)

Table 2.4: Cattle Supertype Entity Attributes

Sr.	Attribute	Data Type	Constraint	Description
01	CattleID *(PK)*	INT	NOT NULL, AUTO_INC, PK	Unique identifier assigned to each animal upon registration.
02	TagNumber	VARCHAR	NOT NULL, UNIQUE	Physical ear-tag number visible on the animal.
03	Breed	VARCHAR	NOT NULL	Recognized breed name (e.g., Sahiwal, Frisian, Cholistani).
04	Gender	ENUM	NOT NULL	Biological sex: 'M' for male (bull) or 'F' for female (cow).
05	DateOfBirth	DATE	NOT NULL	Recorded or estimated birth date of the animal.
06	Status	ENUM	NOT NULL, DEFAULT 'active'	Current life status: 'active', 'sold', or 'dead'.
07	FarmID *(FK)*	INT	NOT NULL	To which farm the cattle belongs.

Cow (Subtype)

Table 2.5: Cow Subtype Entity Attributes

Sr.	Attribute	Data Type	Constraint	Description
01	CattleID(CowID) *(PK/FK)*	INT	NOT NULL, PK, FK $\rightarrow$ CattleID	Inherits CattleID. Serves as both primary and foreign key.
02	LactationStatus	ENUM	NOT NULL, DEFAULT 'dry'	Current milk production status: 'milking' or 'dry'.
03	LastCalvingDate	DATE	NULL	Date of the most recent calving event for this cow.

Bull (Subtype)

Table 2.6: Bull Subtype Entity Attributes

Sr.	Attribute	Data Type	Constraint	Description
01	BullID *(PK/FK)*	INT	NOT NULL, PK, FK → CattleID	Inherits CattleID. Serves as both primary and foreign key.
02	FertilityStatus	VARCHAR	NOT NULL	Indicates the fertility condition of the bull (e.g., High, Moderate, Low).
03	SemenQuality	VARCHAR	NOT NULL	Represents the quality grade of semen used for breeding purposes.

Breeding Record

Table 2.7: Breeding Record Entity Attributes

Sr.	Attribute	Data Type	Constraint	Description
01	BreedingID *(PK)*	INT	NOT NULL, AUTO_INC, PK	Unique identifier for each breeding event.
02	CowID *(FK)*	INT	NOT NULL, FK	References the cow involved in the breeding event.
03	BullID *(FK)*	INT	NOT NULL, FK	References the bull used for breeding.
04	BreedingDate	DATE	NOT NULL	Calendar date on which the breeding event occurred.
05	Method	ENUM	NOT NULL	Breeding method used: ‘natural’ or ‘AI’ (Artificial Insemination).
06	ExpectedDueDate	DATE	NULL	Estimated due date for the cow after successful breeding.
07	Outcome	VARCHAR(50)	NULL	Result of breeding such as ‘Pregnant’, ‘Failed’, or ‘Pending’.
08	Notes	TEXT	NULL	Optional veterinary or farmer notes regarding the breeding event.

Birth Record

Table 2.8: Birth Record Entity Attributes

Sr.	Attribute	Data Type	Constraint	Description
01	BirthID *(PK)*	INT	NOT NULL, AUTO_INC, PK	Unique identifier for each birth event.
02	MotherID *(FK)*	INT	NOT NULL, FK	References the mother cow that gave birth to the calf.
03	CalfID *(FK)*	INT	NOT NULL, FK	References the newborn calf associated with the birth record.
04	BreedingID *(FK)*	INT	NOT NULL, FK	References the breeding event linked to this birth.
05	BirthDate	DATE	NOT NULL	Date on which the calf was born.
06	BirthWeight	DECIMAL	NULL	Weight of the calf at birth in kilograms.
07	Complications	TEXT	NULL	Notes on any birth complications observed during delivery.

Death Record

Table 2.9: Death Record Entity Attributes

Sr.	Attribute	Data Type	Constraint	Description
01	DeathID *(PK)*	INT	NOT NULL, AUTO_INC, PK	Unique identifier for each death record.
02	CattleID *(FK)*	INT	NOT NULL, FK	References the cattle animal associated with the death record.
03	DeathDate	DATE	NOT NULL	Date on which the cattle animal died.
04	Cause	VARCHAR(100)	NULL	Recorded cause of death such as disease, injury, or old age.
05	VetVerified	BOOLEAN	NOT NULL, DEFAULT FALSE	Indicates whether the death record has been verified by a veterinarian.

Purchase Record

Table 2.10: Purchase Record Entity Attributes

Sr.	Attribute	Data Type	Constraint	Description
01	PurchaseID *(PK)*	INT	NOT NULL, AUTO_INC, PK	Unique identifier for each cattle purchase record.
02	CattleID *(FK)*	INT	NOT NULL, FK	References the purchased cattle animal in the CATTLE table.
03	UserID *(FK)*	INT	NOT NULL, FK	References the user who recorded or managed the purchase transaction.
04	SellerName	VARCHAR	NOT NULL	Name of the person or vendor from whom the cattle was purchased.
05	PurchaseDate	DATE	NOT NULL	Date on which the cattle animal was purchased.
06	PurchasePrice	DECIMAL	NOT NULL	Amount paid for purchasing the cattle animal, in Pakistani Rupees.
07	MarketName	VARCHAR	NULL	Name of the livestock market or location where the purchase took place.

Sale Record

Table 2.11: Sale Record Entity Attributes

Sr.	Attribute	Data Type	Constraint	Description
01	SaleID *(PK)*	INT	NOT NULL, AUTO_INC, PK	Unique identifier for each cattle sale transaction.
02	CattleID *(FK)*	INT	NOT NULL, FK	References the cattle animal sold from the CATTLE table.
03	UserID *(FK)*	INT	NOT NULL, FK	References the user who recorded or managed the sale transaction.
04	BuyerName	VARCHAR	NOT NULL	Name of the person or entity who purchased the cattle animal.
05	SaleDate	DATE	NOT NULL	Date on which the cattle animal was sold.
06	SalePrice	DECIMAL	NOT NULL	Amount received from selling the cattle animal, in Pakistani Rupees.
07	MarketName	VARCHAR	NULL	Name of the livestock market or location where the sale took place.

Weight Record

Table 2.12: Weight Record Entity Attributes

Sr.	Attribute	Data Type	Constraint	Description
01	WeightID *(PK)*	INT	NOT NULL, AUTO_INC, PK	Unique identifier for each weight measurement record.
02	CattleID *(FK)*	INT	NOT NULL, FK	References the cattle animal whose weight is being recorded.
03	WeightDate	DATE	NOT NULL	Date on which the animal's weight was measured.
04	WeightKg	DECIMAL	NOT NULL	Weight of the cattle animal in kilograms at the time of measurement.

Color Record

Table 2.13: Color Record Entity Attributes

Sr.	Attribute	Data Type	Constraint	Description
01	ColorID *(PK)*	INT	NOT NULL, AUTO_INC, PK	Unique identifier for each cattle color record.
02	CattleID *(FK)*	INT	NOT NULL, FK	References the cattle animal whose color details are being recorded.
03	PrimaryColor	VARCHAR	NOT NULL	Main coat color of the cattle animal (e.g., white, black, brown).
04	SecondaryColor	VARCHAR	NULL	Additional coat color if the animal has mixed coloring or markings.
05	Pattern	VARCHAR	NULL	Description of the coat pattern such as spotted, solid, or patchy.

Profit/Loss Record

Table 2.14: Profit/Loss Record Entity Attributes

Sr.	Attribute	Data Type	Constraint	Description
01	RecordID *(PK)*	INT	NOT NULL, AUTO_INC, PK	Unique identifier for each profit or loss record.
02	CattleID *(FK)*	INT	NOT NULL, FK	References the cattle animal associated with the financial calculation.
03	PurchaseID *(FK)*	INT	NOT NULL, FK	References the related purchase transaction record.
04	SaleID *(FK)*	INT	NOT NULL, FK	References the related sale transaction record.
05	ProfitLoss	DECIMAL	NOT NULL	Net profit or loss calculated from the cattle transaction.
06	CalculationDate	DATE	NOT NULL	Date on which the profit or loss was calculated.

2.3 BUSINESS RULES

The following business rules define the operational constraints and behavioral policies enforced by the CattleCare Pro database system.

BR1. A **Farm** must be associated with exactly one **User** (administrator). A User may manage one or more Farms.

BR2. A **Farm** must house at least one **Cattle** animal at any point in time. Each Cattle animal belongs to exactly one Farm.

BR3. Every **Cattle** animal must be classified as either a **Cow** or a **Bull**. The specialization is exclusive and total – an animal cannot belong to both subtypes simultaneously.

BR4. A **Cattle** animal may have at most one **Death Record**. Once a death is recorded, the animal's Status must be updated to 'dead'.

BR5. A **Cattle** animal may have at most one **Purchase Record**, reflecting the initial acquisition of the animal.

BR6. A **Cattle** animal may have at most one **Sale Record**. Once sold, the animal's Status must be updated to 'sold'.

BR7. A **Cattle** animal may have multiple **Weight Records** over its lifetime.

BR8. A **Cattle** animal may have multiple **Color Records** over its lifetime.

BR9. A **Cow** may participate in zero or more **Breeding Records** as the dam. A **Bull** may participate in zero or more Breeding Records as the sire.

BR10. Each **Breeding Record** must be linked to exactly one **Cow** and exactly one **Bull**.

BR11. A **Breeding Record** may result in at most one **Birth Record**. Not every breeding event leads to a successful birth.

BR12. A **Birth Record** must be linked to exactly one **Breeding Record** and must produce exactly one new Cattle record representing the calf.

BR13. Each **Profit/Loss Record** must be associated with exactly one Cattle animal and is computed only when both a Purchase Record and a Sale Record exist.

BR14. A **Purchase Record** may contribute to at most one **Profit/Loss Record**. Similarly, a Sale Record may contribute to at most one Profit/Loss Record.

BR15. The ProfitLossAmount is calculated as:

$$\text{ProfitLossAmount} = \text{SalePrice} - \text{PurchasePrice} \tag{2.1}$$

A positive value represents profit; a negative value represents a loss.

BR16. A **User** must have a unique email address. No two users may share the same email.

BR17. The **TagNumber** of every Cattle animal must be unique across the entire system.

2.4 ENTITY RELATIONSHIP DIAGRAM

The Enhanced Entity-Relationship Diagram (EERD) below represents the complete structural design of the CattleCare Pro database. The diagram uses Crow's Foot notation to illustrate all entities, their attributes, the specialisation hierarchy (Cattle → Cow / Bull), and all relationships with their cardinalities.

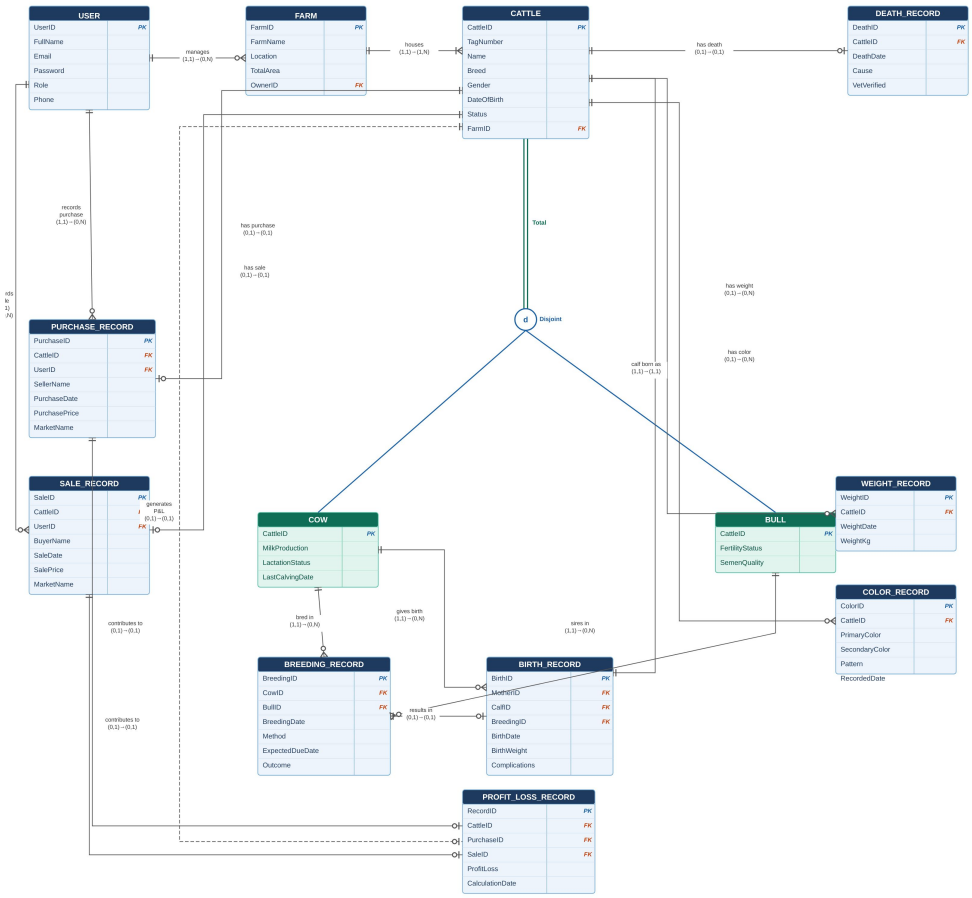

Figure 2.1: Enhanced Entity-Relationship Diagram of CattleCare Pro using Crow's Foot Notation.

The following table summarises all relationships and cardinalities as represented in the EERD.

Table 2.15: Relationship Summary with Cardinalities

From Entity	**Relationship**	**To Entity**	**Cardinality**
User	manages	Farm	(1,1) to (0,N)
Farm	houses	Cattle	(1,1) to (1,N)
Cattle	has death	Death Record	(1,1) to (0,1)
Cattle	has purchase	Purchase Record	(1,1) to (0,1)
Cattle	has sale	Sale Record	(1,1) to (0,1)
Cattle	has weight	Weight Record	(1,1) to (0,N)
Cattle	has color	Color Record	(1,1) to (0,N)
Cattle	generates	Profit/Loss Rec.	(1,1) to (0,1)
Cow	bred in	Breeding Record	(1,1) to (0,N)
Bull	sires in	Breeding Record	(1,1) to (0,N)
Breeding Record	results in	Birth Record	(1,1) to (0,1)
Cow	gives birth to	Birth Record	(1,1) to (0,N)
Birth Record	calf born as	Cattle	(1,1) to (1,1)
Purchase Rec.	contributes to	Profit/Loss Rec.	(1,1) to (0,1)
Sale Record	contributes to	Profit/Loss Rec.	(1,1) to (0,1)

CHAPTER 3: Logical Database Design

3.1 RELATIONAL SCHEMA

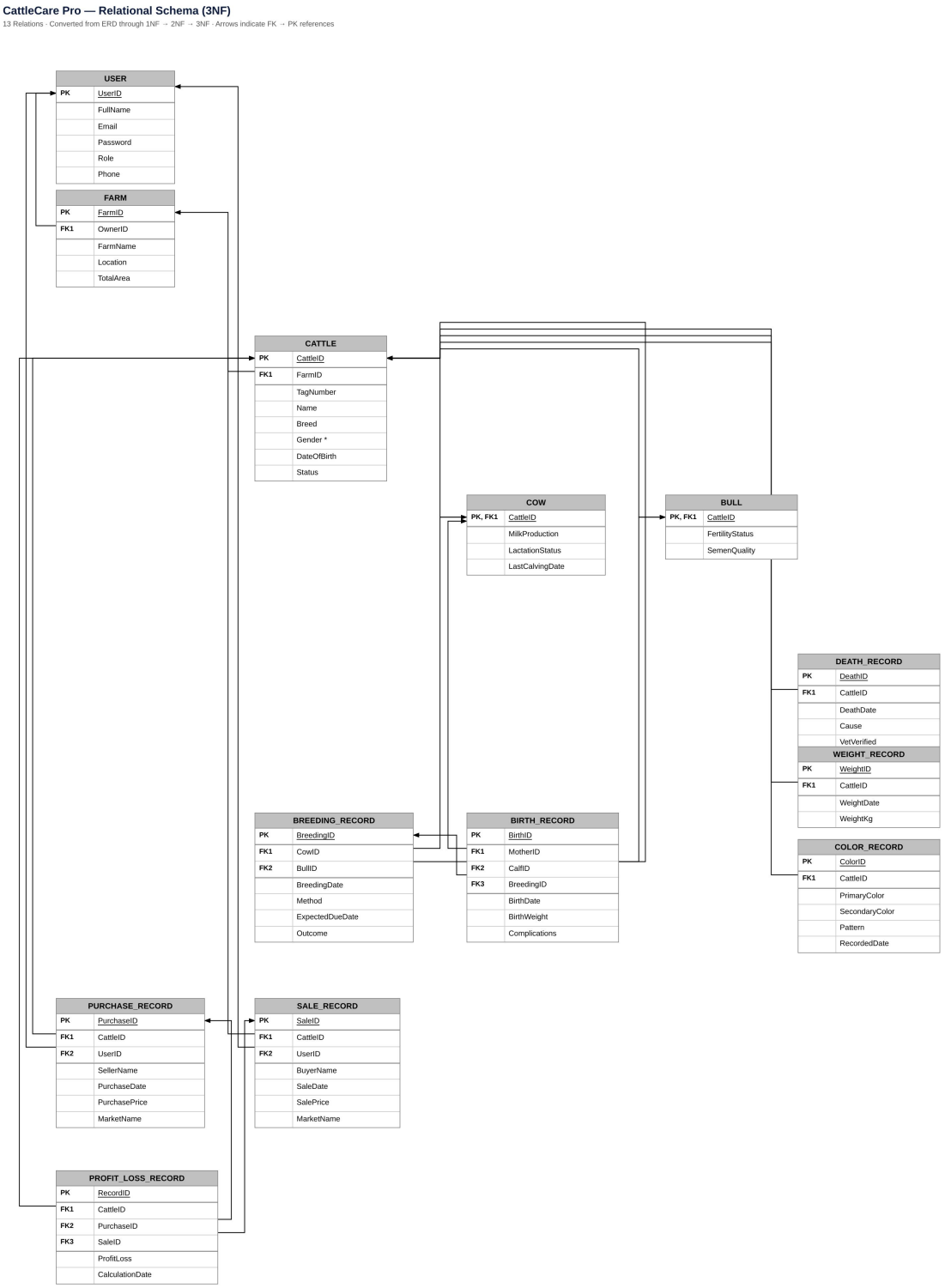

Figure 3.1: Relational Schema of CattleCare Pro Database

3.2 FUNCTIONAL DEPENDENCIES

USER

Functional Dependency:

[UserID $\rightarrow FullName, Email, Password, Role, Phone$]

Example:

UserID = 1 determines:

(FullName = Muhammad Ali, Email = ali@gmail.com, Role = Admin)

FARM

Functional Dependency:

[FarmID $\rightarrow FarmName, Location, TotalArea, OwnerID$]

Example:

FarmID = 101 determines:

(FarmName = Green Farm, Location = Mianwali, OwnerID = 1)

CATTLE

Functional Dependency:

[CattleID $\rightarrow TagNumber, Name, Breed, Gender, DateOfBirth, Status, FarmID$]

Example:

CattleID = 1001 determines all cattle information.

COW

Functional Dependency:

[CattleID $\rightarrow MilkProduction, LactationStatus, LastCalvingDate$]

Example:

CattleID = 1001 determines:

(MilkProduction = 12L, LactationStatus = Lactating)

BULL

Functional Dependency:

[CattleID $\rightarrow FertilityStatus, SemenQuality$]

Example:

CattleID = 2001 determines:

(FertilityStatus = Fertile, SemenQuality = Excellent)

DEATH_RECORD

Functional Dependency:

[DeathID $\rightarrow CattleID, DeathDate, Cause, VetVerified$]

Example:

DeathID = 301 determines the complete death record.

WEIGHT_RECORD

Functional Dependency:

[WeightID $\rightarrow CattleID, WeightDate, WeightKg$]

Example:

WeightID = 401 determines:

(CattleID = 1001, WeightKg = 420)

COLOR_RECORD

Functional Dependency:

[ColorID $\rightarrow CattleID, PrimaryColor, SecondaryColor, Pattern, RecordedDate$]

Example:

ColorID = 501 determines all colour details.

BREEDING_RECORD

Functional Dependency:

[BreedingID $\rightarrow CowID, BullID, BreedingDate, Method, ExpectedDueDate, Outcome$]

Example:

BreedingID = 601 determines the complete breeding event.

BIRTH_RECORD

Functional Dependency:

[BirthID $\rightarrow MotherID, CalfID, BreedingID, BirthDate, BirthWeight, Complications$]

Example:

BirthID = 701 determines all birth details.

PURCHASE_RECORD

Functional Dependency:

[PurchaseID $\rightarrow CattleID, UserID, SellerName, PurchaseDate, PurchasePrice, MarketName$]

Example:

PurchaseID = 801 determines the complete purchase transaction.

SALE_RECORD

Functional Dependency:

[SaleID $\rightarrow CattleID, UserID, BuyerName, SaleDate, SalePrice, MarketName$]

Example:

SaleID = 901 determines the complete sale transaction.

PROFIT_LOSS_RECORD

Functional Dependency:

[RecordID $\rightarrow CattleID, PurchaseID, SaleID, ProfitLoss, CalculationDate$]

Example:

RecordID = 1001 determines the complete profit/loss record.

3.3 NORMALIZATION

Partial Functional Dependencies

During normalization, partial dependencies were identified in the unnormalized data where some attributes depended only on part of a composite key rather than the whole key.

Examples include:

[CattleID $\rightarrow TagNumber, Name, Breed, Gender, DateOfBirth, Status$]

[FarmID $\rightarrow FarmName, Location, TotalArea$]

[UserID $\rightarrow FullName, Email, Password, Role, Phone$]

These dependencies indicated that several attributes were related to only one part of the original data structure and therefore needed to be separated into individual relations.

Transitive Functional Dependencies

Transitive dependencies were analyzed to ensure that non-key attributes did not determine other non-key attributes.

Examples checked during normalization:

[PurchaseID $\rightarrow MarketName$]

[SaleID $\rightarrow MarketName$]

The schema was examined for possible transitive dependencies, and no significant violations were found. Therefore, no additional decomposition was required beyond the identified relations.

Normalization Process

First Normal Form (1NF)

In the original ERD, all entities were examined to ensure that their attributes contained only atomic values. Each attribute stores a single indivisible value and no multi-valued attributes or repeating columns exist within any entity. Therefore, the database design satisfies the requirements of First Normal Form (1NF) directly from the ERD structure without requiring any decomposition. Examples of atomic attributes include FullName, Email, FarmName, Breed, WeightKg, and BirthDate, where each field stores only a single value for a particular record.

Second Normal Form (2NF)

Partial dependencies were removed by separating attributes into relations based on their primary keys. Entity-specific information was stored in independent tables such as USER, FARM, CATTLE, COW, and BULL.

This ensured that every non-key attribute depended on the entire primary key of its relation.

Third Normal Form (3NF)

All relations were examined for transitive dependencies. Non-key attributes were verified to depend only on their respective primary keys. No additional decomposition was required because the resulting schema already satisfied Third Normal Form.

As a result, redundancy was minimized and insertion, deletion, and update anomalies were eliminated.

Final 3NF Relations

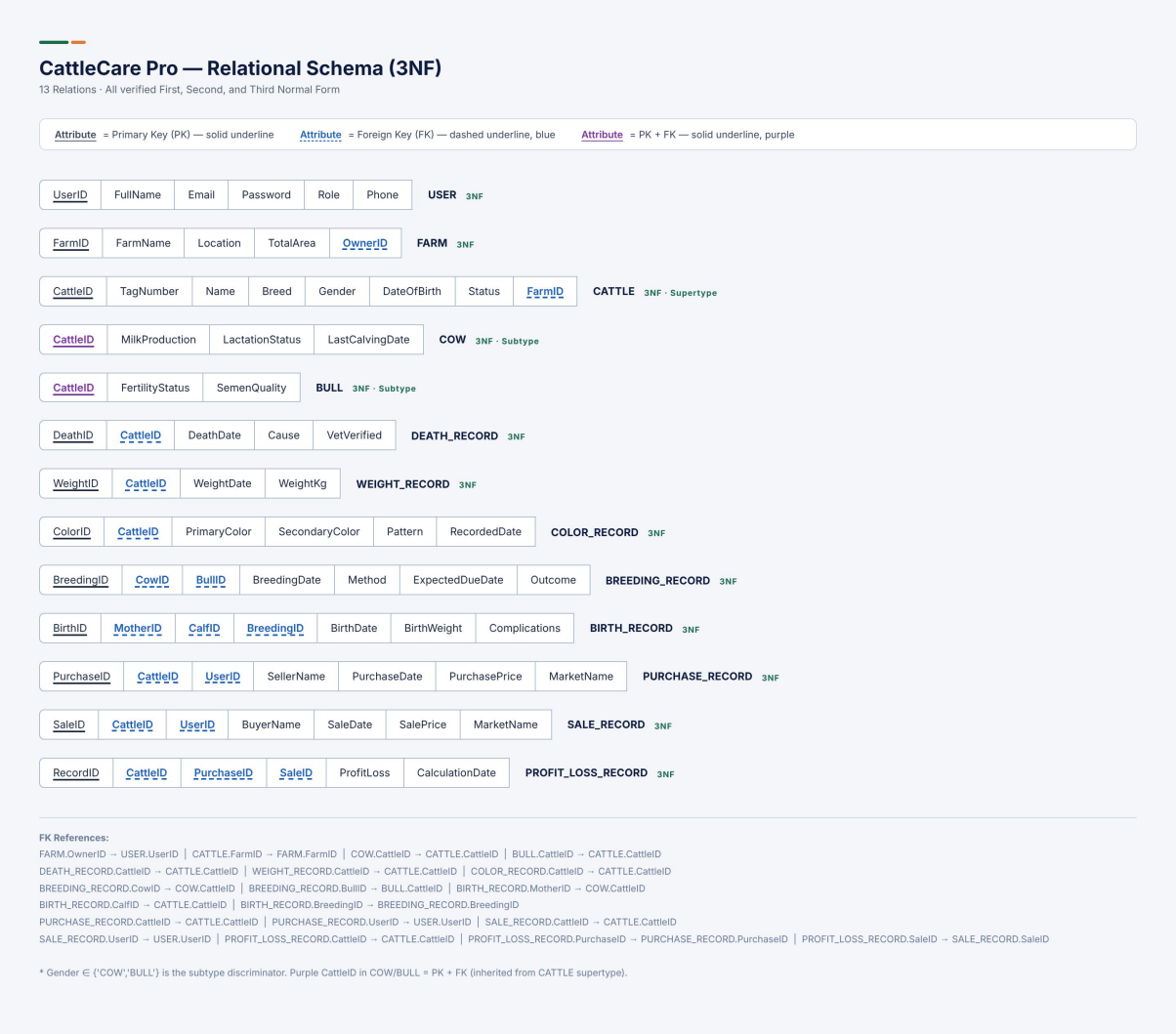

Figure 3.2: Relational Schema of CattleCare Pro Database

Therefore, all thirteen relations of the CattleCare Pro database are in Third Normal Form (3NF).

CHAPTER 4: Implementation

4.1 LANGUAGE/FRAMEWORK

The CattleCare Pro system is implemented using the following technology stack:

Frontend (GUI): React.js is used to build the user interface. React was selected for its component-based architecture, which allows modular development of UI components such as cattle dashboards, breeding forms, weight tracking panels, and financial summaries. React's virtual DOM ensures efficient UI updates when farm data changes.

Backend (REST API): Node.js with the Express.js framework serves as the server-side layer. Express provides a lightweight and flexible REST API that the React frontend consumes via HTTP requests. All business logic and database query execution are handled at this layer.

Database: MySQL 9.6 is the chosen relational database management system. It was selected for its robustness, support for stored procedures, triggers, and views, and widespread use in production environments. The `mysql2` npm package is used as the Node.js MySQL driver.

Database Connectivity: The Express.js backend connects to MySQL using the `mysql2` library configured with a connection pool. The React frontend communicates exclusively with the Express API using the Axios HTTP client; it has no direct access to the database.

4.2 DATABASE CONNECTIVITY

The application follows a three-tier architecture:

1. **Presentation Tier:** React.js application running in the browser. Users interact with forms and dashboards. All data requests are sent as HTTP calls to the Express API.
2. **Application Tier:** Node.js/Express.js REST API server. It receives HTTP requests, validates inputs, executes SQL queries or calls stored procedures against the MySQL database, and returns JSON responses.
3. **Data Tier:** MySQL 9.6 database server (`cattlecare_pro` schema). All persistent data is stored here. Triggers enforce business rules automatically on insert events.

Connection parameters used in the Node.js backend:

- **Host:** localhost
- **Port:** 3306
- **Database:** cattlecare_pro
- **Driver:** mysql2 (npm package)
- **Connection method:** Connection pool (`mysql2.createPool`)

4.3 STORED PROCEDURES AND FUNCTIONS

The following database programmability objects were developed for the CattleCare Pro system. They encapsulate core data operations and ensure consistent, validated interactions with the

database.

Table 4.1: Stored Procedures, Functions, Triggers, and Views

#	Object Name	Type	Description	Input Parameters	Output / Return
1	sp_AddCattle	Stored Procedure	Inserts a new cattle animal record into the CATTLE table with status 'Active'.	TagNumber, Name, Breed, Gender, DateOfBirth, FarmID	OUT NewCattleID (INT)
2	sp_AddCow	Stored Procedure	Inserts a new record into the COW subtype table for a female cattle.	CattleID, MilkProd, LactStatus, LastCalving	None
3	sp_AddBull	Stored Procedure	Inserts a new record into the BULL subtype table for a male cattle.	CattleID, FertilityStatus, SemenQuality	None
4	sp_RecordBreeding	Stored Procedure	Creates a new breeding event record linking a cow and a bull.	CowID, BullID, BreedingDate, Method, ExpectedDue	None
5	sp_RecordBirth	Stored Procedure	Records a birth event, updates the mother's LastCalvingDate, and marks the breeding outcome as Successful.	MotherID, CalfID, BreedingID, BirthDate, BirthWeight, Complications	None
6	sp_RecordPurchase	Stored Procedure	Inserts a cattle purchase record including seller details and market information.	CattleID, UserID, SellerName, PurchaseDate, Price, MarketName	None
7	sp_RecordSale	Stored Procedure	Inserts a cattle sale record. The trg_after_sale_insert and trg_auto_profitloss_on_sale triggers fire automatically after this insert.	CattleID, UserID, BuyerName, SaleDate, Price, MarketName	None

#	Object Name	Type	Description	Input Parameters	Output / Return
8	sp_RecordDeath	Stored Procedure	Inserts a death record. The trg_after_death_insert trigger fires automatically to set the animal's status to 'Dead'.	CattleID, DeathDate, Cause, VetVerified	None
9	sp_AddWeightRecord	Stored Procedure	Records a periodic weight measurement for a cattle animal.	CattleID, Date, WeightKg	None
10	sp_GetCattleProfile	Stored Procedure	Retrieves full profile of a cattle animal: basic info, farm name, all weight records (newest first), and all colour records (newest first).	CattleID	Three SELECT result sets
11	trg_after_death_insert	Trigger (AFTER INSERT)	Automatically sets the Status of the cattle animal to 'Dead' in the CATTLE table when a death record is inserted.	Implicit: NEW.CattleID	Updates CATTLE.Status
12	trg_after_sale_insert	Trigger (AFTER INSERT)	Automatically sets the Status of the cattle animal to 'Sold' in the CATTLE table when a sale record is inserted.	Implicit: NEW.CattleID	Updates CATTLE.Status
13	trg_auto_profitloss_on_sale	Trigger (AFTER INSERT)	Automatically calculates and inserts a Profit/Loss Record when a sale is recorded. Looks up the most recent purchase price for the animal and computes SalePrice − PurchasePrice.	NEW.CattleID, NEW.SalePrice, NEW.SaleID	Inserts into PROFIT_LOSS_RECORD
14	trg_check_gender_cow	Trigger (BEFORE INSERT)	Prevents insertion into the COW table if the referenced cattle animal's gender is not Female. Raises a custom SQL error on violation.	Implicit: NEW.CattleID	Error signal or allow insert
15	trg_check_gender_bull	Trigger (BEFORE INSERT)	Prevents insertion into the BULL table if the referenced cattle animal's gender is not Male. Raises a custom SQL error on violation.	Implicit: NEW.CattleID	Error signal or allow insert

#	Object Name	Type	Description	Input Parameters	Output / Return
16	vw_ActiveCattle	View	Returns a list of all currently active (Status = 'Active') cattle animals joined with their farm name.	None	CattleID, TagNumber, Name, Breed, Gender, DateOfBirth, Status, FarmName
17	vw_CowMilkOverview	View	Returns milk production details for all active cows: tag number, name, daily milk output, lactation status, and last calving date.	None	CattleID, TagNumber, Name, MilkProduction, LactationStatus, LastCalvingDate
18	vw_ProfitLossSummary	View	Provides a financial summary per animal: tag number, cattle name, purchase price, sale price, profit/loss amount, and calculation date.	None	RecordID, TagNumber, CattleName, PurchasePrice, SalePrice, ProfitLoss, CalculationDate

4.3.1 Key DDL Excerpts

The following excerpts from `dbDDL.sql` illustrate the most significant implementation details of the database schema.

Core Table Definitions

```
CREATE TABLE USER (
    UserID    INT NOT NULL AUTO_INCREMENT,
    FullName  VARCHAR(100) NOT NULL,
    Email     VARCHAR(150) NOT NULL UNIQUE,
    Password  VARCHAR(255) NOT NULL,
    Role      ENUM('Admin','Manager','Vet','Viewer') NOT NULL
       DEFAULT 'Viewer',
    Phone     VARCHAR(20),
    CONSTRAINT pk_user PRIMARY KEY (UserID)
);
```

```
10
11 CREATE TABLE FARM (
12     FarmID    INT NOT NULL AUTO_INCREMENT,
13     FarmName  VARCHAR(150) NOT NULL,
14     Location  VARCHAR(255),
15     TotalArea DECIMAL(10,2),
16     OwnerID   INT NOT NULL,
17     CONSTRAINT pk_farm      PRIMARY KEY (FarmID),
18     CONSTRAINT fk_farm_user FOREIGN KEY (OwnerID)
19         REFERENCES USER(UserID) ON UPDATE CASCADE ON DELETE
               RESTRICT
20 );
```

Listing 4.1: USER and FARM table definitions

```
1  CREATE TABLE CATTLE (
2      CattleID    INT NOT NULL AUTO_INCREMENT,
3      TagNumber   VARCHAR(50) NOT NULL UNIQUE,
4      Name        VARCHAR(100),
5      Breed       VARCHAR(100),
6      Gender      ENUM('Male','Female') NOT NULL,
7      DateOfBirth DATE,
8      Status      ENUM('Active','Sold','Dead') NOT NULL DEFAULT
           'Active',
9      FarmID      INT NOT NULL,
10     CONSTRAINT pk_cattle      PRIMARY KEY (CattleID),
11     CONSTRAINT fk_cattle_farm FOREIGN KEY (FarmID)
12         REFERENCES FARM(FarmID) ON UPDATE CASCADE ON DELETE
               RESTRICT
13 );
14
15 CREATE TABLE COW (
16     CattleID        INT NOT NULL,
17     MilkProduction  DECIMAL(6,2),
18     LactationStatus ENUM('Lactating','Dry','Pregnant') NOT NULL
           DEFAULT 'Dry',
19     LastCalvingDate DATE,
20     CONSTRAINT pk_cow        PRIMARY KEY (CattleID),
21     CONSTRAINT fk_cow_cattle FOREIGN KEY (CattleID)
22         REFERENCES CATTLE(CattleID) ON UPDATE CASCADE ON DELETE
               CASCADE
23 );
24
25 CREATE TABLE BULL (
26     CattleID       INT NOT NULL,
27     FertilityStatus ENUM('Fertile','Infertile','Unknown') NOT
           NULL DEFAULT 'Unknown',
28     SemenQuality    ENUM('Excellent','Good','Poor','Unknown')
           NOT NULL DEFAULT 'Unknown',
29     CONSTRAINT pk_bull        PRIMARY KEY (CattleID),
30     CONSTRAINT fk_bull_cattle FOREIGN KEY (CattleID)
31         REFERENCES CATTLE(CattleID) ON UPDATE CASCADE ON DELETE
```

```
            CASCADE
);
```

Listing 4.2: CATTLE supertype and COW/BULL subtype table definitions

Trigger Definitions

```
DELIMITER $$
CREATE TRIGGER trg_after_death_insert
AFTER INSERT ON DEATH_RECORD
FOR EACH ROW
BEGIN
    UPDATE CATTLE SET Status = 'Dead' WHERE CattleID =
       NEW.CattleID;
END$$
DELIMITER ;
```

Listing 4.3: Trigger: automatically set Status to Dead on death record insert

```
DELIMITER $$
CREATE TRIGGER trg_auto_profitloss_on_sale
AFTER INSERT ON SALE_RECORD
FOR EACH ROW
BEGIN
    DECLARE v_purchaseID    INT DEFAULT NULL;
    DECLARE v_purchasePrice DECIMAL(12,2) DEFAULT 0;

    SELECT PurchaseID, PurchasePrice
    INTO   v_purchaseID, v_purchasePrice
    FROM   PURCHASE_RECORD
    WHERE  CattleID = NEW.CattleID
    ORDER BY PurchaseDate DESC LIMIT 1;

    INSERT INTO PROFIT_LOSS_RECORD
        (CattleID, PurchaseID, SaleID, ProfitLoss,
           CalculationDate)
    VALUES
        (NEW.CattleID, v_purchaseID, NEW.SaleID,
         NEW.SalePrice - v_purchasePrice, CURDATE());
END$$
DELIMITER ;
```

Listing 4.4: Trigger: automatically compute and insert Profit/Loss on sale

```
DELIMITER $$
CREATE TRIGGER trg_check_gender_cow
BEFORE INSERT ON COW
FOR EACH ROW
BEGIN
    DECLARE v_gender VARCHAR(10);
    SELECT Gender INTO v_gender FROM CATTLE WHERE CattleID =
       NEW.CattleID;
```

```
    IF v_gender <> 'Female' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot add COW record: cattle
           gender is not Female';
    END IF;
END$$
DELIMITER ;

DELIMITER $$
CREATE TRIGGER trg_check_gender_bull
BEFORE INSERT ON BULL
FOR EACH ROW
BEGIN
    DECLARE v_gender VARCHAR(10);
    SELECT Gender INTO v_gender FROM CATTLE WHERE CattleID =
       NEW.CattleID;
    IF v_gender <> 'Male' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot add BULL record: cattle
           gender is not Male';
    END IF;
END$$
DELIMITER ;
```

Listing 4.5: Triggers: enforce gender constraints on COW and BULL inserts

View Definitions

```
CREATE OR REPLACE VIEW vw_ActiveCattle AS
SELECT c.CattleID, c.TagNumber, c.Name, c.Breed, c.Gender,
       c.DateOfBirth, c.Status, f.FarmName
FROM CATTLE c
JOIN FARM f ON c.FarmID = f.FarmID
WHERE c.Status = 'Active';
```

Listing 4.6: View: active cattle overview

```
CREATE OR REPLACE VIEW vw_ProfitLossSummary AS
SELECT pl.RecordID, c.TagNumber, c.Name AS CattleName,
       pr.PurchasePrice, sr.SalePrice,
       pl.ProfitLoss, pl.CalculationDate
FROM PROFIT_LOSS_RECORD pl
JOIN CATTLE c ON pl.CattleID = c.CattleID
LEFT JOIN PURCHASE_RECORD pr ON pl.PurchaseID = pr.PurchaseID
LEFT JOIN SALE_RECORD     sr ON pl.SaleID     = sr.SaleID;
```

Listing 4.7: View: profit/loss summary per animal

Stored Procedure Example

```
DELIMITER $$
CREATE PROCEDURE sp_RecordBirth(
```

```
    IN p_MotherID    INT,
    IN p_CalfID      INT,
    IN p_BreedingID  INT,
    IN p_BirthDate   DATE,
    IN p_BirthWeight DECIMAL(5,2),
    IN p_Complications TEXT
)
BEGIN
    -- Insert the birth record
    INSERT INTO BIRTH_RECORD
        (MotherID, CalfID, BreedingID, BirthDate, BirthWeight,
           Complications)
    VALUES
        (p_MotherID, p_CalfID, p_BreedingID,
         p_BirthDate, p_BirthWeight, p_Complications);

    -- Update the mother's last calving date
    UPDATE COW SET LastCalvingDate = p_BirthDate WHERE CattleID
       = p_MotherID;

    -- Mark the breeding event as Successful
    IF p_BreedingID IS NOT NULL THEN
        UPDATE BREEDING_RECORD
        SET Outcome = 'Successful'
        WHERE BreedingID = p_BreedingID;
    END IF;
END$$
DELIMITER ;
```

Listing 4.8: Stored procedure: sp_RecordBirth

Performance Indexes

```
CREATE INDEX idx_cattle_farm    ON CATTLE(FarmID);
CREATE INDEX idx_cattle_status  ON CATTLE(Status);
CREATE INDEX idx_cattle_gender  ON CATTLE(Gender);
CREATE INDEX idx_weight_cattle  ON WEIGHT_RECORD(CattleID);
CREATE INDEX idx_weight_date    ON WEIGHT_RECORD(WeightDate);
CREATE INDEX idx_breeding_cow   ON BREEDING_RECORD(CowID);
CREATE INDEX idx_breeding_date  ON
   BREEDING_RECORD(BreedingDate);
CREATE INDEX idx_birth_mother   ON BIRTH_RECORD(MotherID);
CREATE INDEX idx_purchase_date  ON
   PURCHASE_RECORD(PurchaseDate);
CREATE INDEX idx_sale_date      ON SALE_RECORD(SaleDate);
```

Listing 4.9: Index definitions for query optimisation

“‘latex

4.4 INTERFACES

[This section includes screenshots of the implemented React GUI interfaces.]

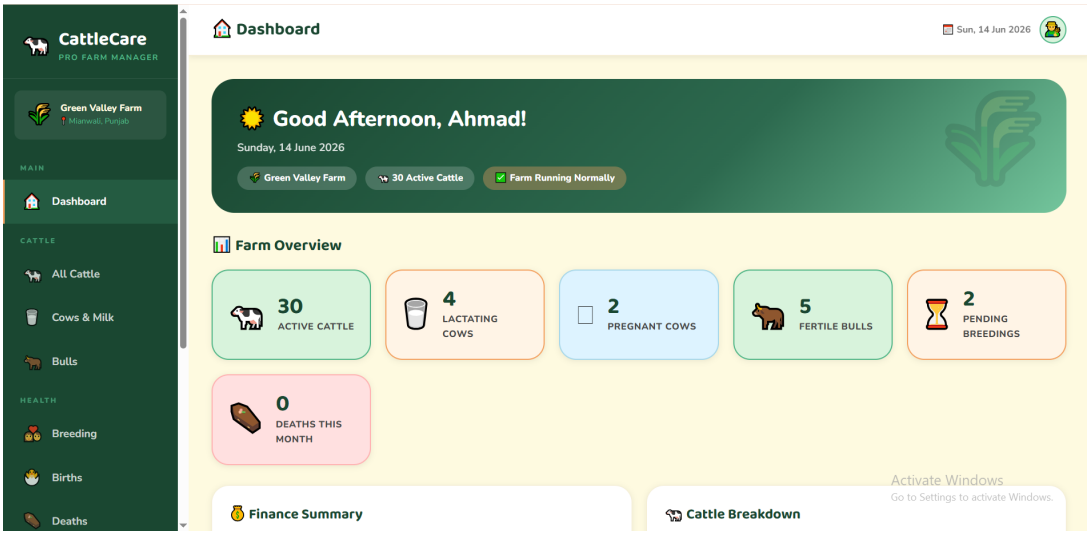

Figure 4.1: Dashboard – Active Cattle Overview

Description: The main dashboard displays number of currently active cattle, Lactating Cows, Pregnant Cows, Fertile Bulls, Pending Breedings, and Deaths this Month, along with the financial summary following the above portion.

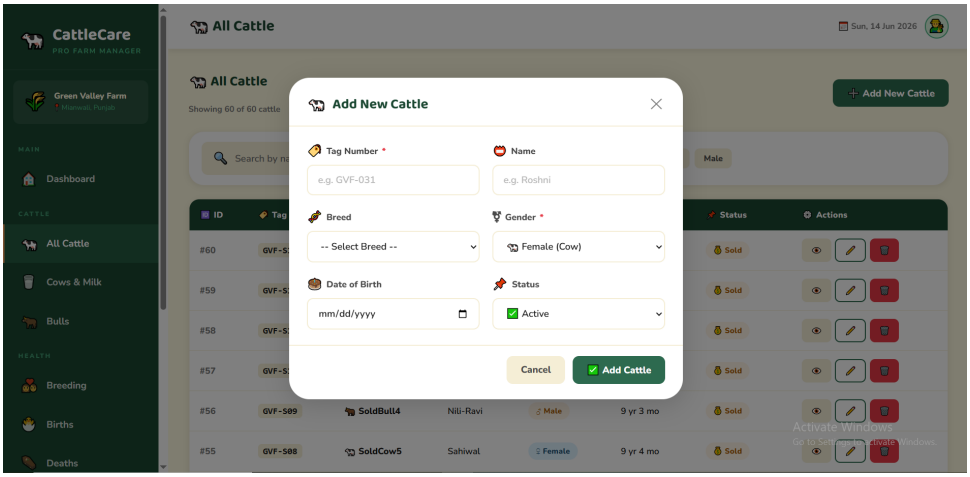

Figure 4.2: Cattle Registration Form

Description: A React form allowing the admin to register new cattle. On submission, the form calls the `sp\_AddCattle` stored procedure via the Express API.

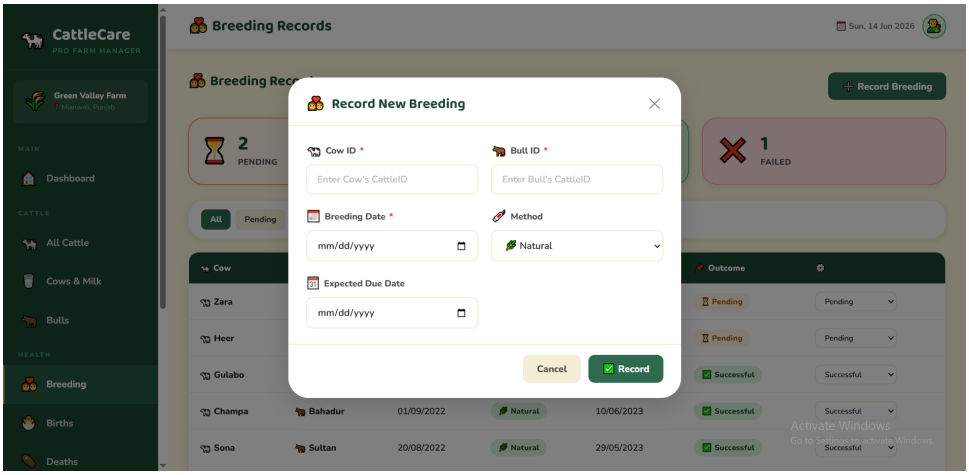

Figure 4.3: Breeding Record Entry

Description: A form to record a breeding event by selecting a cow and bull IDs. Calls `sp\_RecordBreeding` on the backend.

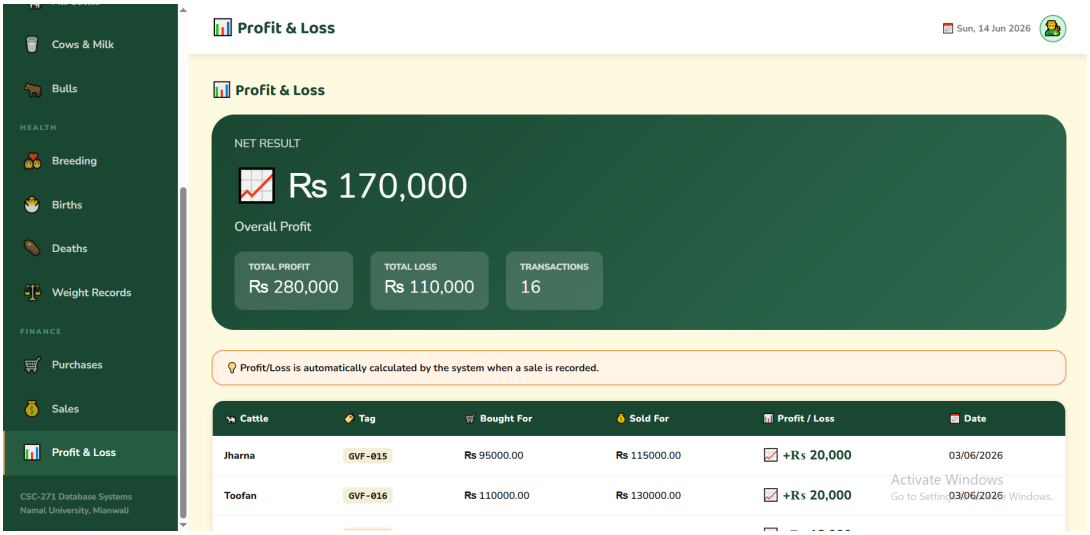

Figure 4.4: Financial Summary – Profit/Loss

Description: A financial dashboard view displaying profit and loss per animal. Data is retrieved from the `vw\_ProfitLossSummary` view. ‘‘‘

References

[1] Department of Computer Science, "Database Systems Course Material," Namal University Mianwali, Spring 2026.

[2] Oracle Corporation, "MySQL 8.0 Reference Manual," Oracle, 2024. [Online]. Available: https://dev.mysql.com/doc/refman/9.6/en/

[3] IEEE, "IEEE Standard for Software Requirements Specifications," IEEE Std 830-1998, 1998.

[4] CattleCare Pro GitHub Repository. [Online]. Available: https://github.com/aligoharcodepulse/cattlecare-pro-db.git

[5] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

[6] R. Elmasri and S. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.

Appendix: AI Usage and Prompts

In accordance with the course guidelines, the following section documents all AI tools and online services used during the preparation of this Milestone 3 report. All AI-generated content was critically reviewed, verified against course material, and adapted by team members to ensure accuracy.

Table 4.2: AI Tools and Prompts Used

Sr.	Tool Used	Prompt / Action	Purpose
01	Claude AI	"Design a complete Enhanced Entity-Relationship Diagram (EERD) for a livestock breeding and farm management system named CattleCare Pro using Crow's Foot notation. Output as an HTML file."	Used to design and generate the complete EERD for the CattleCare Pro system.
02	CloudConvert	Uploaded the HTML EERD file and converted to JPG format.	Converted the HTML-based EERD diagram into a JPG image for inclusion in the report.
03	Claude AI	"Generate a complete Milestone 3 Database Design Document for CattleCare Pro following the provided university template. Format it in LaTeX following IEEE standards."	Used to generate the full structured Milestone 3 report in LaTeX.